

Flujo de Trabajo en GitHub

Creación de Repositorio

Esta fase inicial establece las bases para un desarrollo organizado. Implica definir los archivos base, configuración del entorno y estructura de carpetas.

Nota de estilo

Se recomienda el uso de **minúsculas** como formato estándar para nombres de ramas, documentos y extensiones.

Usar el carácter de subrayado (`_`) para sustituir espacios.

Evitar el uso de **acentos, caracteres especiales o mayúsculas** para mantener la compatibilidad y una distribución uniforme en repositorios y sistemas de control de versiones.

Inicialización

- Crear el repositorio desde GitHub.
- Incluir archivos esenciales como: .gitignore, LICENSE, y README.md.
- Establecer la rama principal (main).
- Configurar reglas de protección de ramas si se requiere seguridad adicional.

Estructura básica del repositorio

El repositorio se organiza en una estructura clara y modular que facilita el mantenimiento, escalabilidad y reutilización del proyecto. A continuación se detalla el significado y uso de cada carpeta y archivo:

Organizar el repositorio en carpetas como

- docs/
 - generado automáticamente
- software/
 - documentation/
 - datasheet/
 - sphinx
 - README.md
 - examples/

- c/
- mp/
- hardware/
 - resources/
 - unit_pinout_v_0_0_x_sku_[name_product]_en.png
 - unit_pinout_v_0_0_x_sku_[name_product]_es.png
 - unit_dimension_v_0_0_x_sku_[name_product].png
 - unit_block_v_0_0_x_sku_[name_product].png
 - unit_topology_v_0_0_x_sku_[name_product].png
 - unit_bottom_v_0_0_x_sku_[name_product].png
 - unit_top_v_0_0_x_sku_[name_product].png
 - unit_power_tree_v_0_0_x_sku_[name_product].png
 - unit_sch_v_0_0_x_sku_[name_product].pdf
 - img/
 - img_sku_[name_product]_1.png
 - img_sku_[name_product]_2.png
 - img_sku_[name_product]_3.png
 - README.md
- README.md
- LICENSE
- .gitignore

docs/

Contiene archivos generados automáticamente, como documentación técnica en formato PDF o HTML que puede ser publicada mediante GitHub Pages.

• unit_product_brief.pdf

Documento técnico compilado desde los archivos fuente, normalmente generado por scripts o flujos de CI/CD.

software/

Incluye todo el software asociado al proyecto, tanto documentación como ejemplos y código fuente.

• documentation/

Scripts de documentación (por ejemplo, convertidores de Markdown a LaTeX o PDF).

- `datasheet/`

Contiene documentación técnica extendida, esquemáticos explicados o datasheets personalizados.

- `sphinx/`

Archivos de configuración y fuentes para generar documentación con Sphinx.

- `README.md`

Instrucciones específicas para desarrollar o compilar el software asociado.

- `examples/`

Código de ejemplo organizado por lenguajes.

- `c_code/`

Ejemplos en lenguaje C para pruebas de hardware o integración con firmware.

- `python_code/`

Scripts en Python para comunicación, visualización o pruebas automatizadas.

 `hardware/`

Alberga los recursos relacionados con el diseño electrónico del producto.

- `resources/`

Archivos visuales y documentación técnica del hardware.

- `img/`

Contiene imágenes y diagramas del producto en distintas vistas o versiones.

- `img_sku_[name_product]_1.png`

Imagen representativa del producto desde una perspectiva frontal.

- `img_sku_[name_product]_2.png`

Vista lateral o alternativa del producto.

- `img_sku_[name_product]_3.png`

Imagen adicional, como un acercamiento o uso en contexto.

- `unit_pinout_v_0_0_x_sku_[name_product]_en.png`

Diagrama de pines en inglés.

- `unit_pinout_v_0_0_x_sku_[name_product]_es.png`

Diagrama de pines en español.

- `unit_block_v_0_0_x_sku_[name_product].png`

Diagrama de bloques funcionales del módulo.

- `unit_topology_v_0_0_x_sku_[name_product].png`
Topología del sistema o interconexiones principales.
 - `unit_botton_v_0_0_x_sku_[name_product].png`
Vista inferior del PCB o del módulo ensamblado.
 - `unit_top_v_0_0_x_sku_[name_product].png`
Vista superior del PCB.
 - `unit_power_tree_v_0_0_x_sku_[name_product].png`
Esquema de distribución de energía.
 - `unit_sch_v_0_0_x_sku_[name_product].pdf`
Esquemático completo del circuito en formato PDF.
-

Archivos raíz

- `README.md`
Descripción general del proyecto, instrucciones de uso, dependencias y créditos.
- `LICENSE`
Licencia de uso y distribución del contenido del repositorio.
- `.gitignore`
Lista de archivos o carpetas que deben ser ignorados por Git (por ejemplo, binarios, temporales, cache).

Crear etiquetas (labels) útiles para clasificar issues y pull requests.

Etiquetas para títulos de issues

1. Fix
2. Docs
3. Error
4. Question
5. Update

 Añadir archivos como CONTRIBUTING.md y CODE_OF_CONDUCT.md para definir normas de colaboración.

Mantenimiento del Repositorio

Una vez creado, el repositorio requiere mantenimiento continuo para garantizar su funcionamiento óptimo y ordenado.

Control de versiones

Usar etiquetas git (git tag) para marcar versiones estables.

Documentar los cambios relevantes en un archivo CHANGELOG.md

Automatización (CI/CD)

Configurar flujos de trabajo automáticos con GitHub Actions.

Automatizar pruebas, linting, compilaciones y despliegues.

Integrar herramientas externas si es necesario (Docker, PyPI, etc.).

Limpieza y orden

Eliminar ramas antiguas que ya no se usan.

Mantener el repositorio libre de archivos innecesarios.

Actualizar la documentación regularmente.

Solicitudes y Gestión Colaborativa

Permite a los colaboradores proponer cambios, reportar errores y participar en el desarrollo.

Issues (Errores o mejoras)

Los Issues permiten reportar errores, proponer nuevas funciones o documentar tareas pendientes. Cada Issue debe ser claro, específico y seguir el formato preestablecido para facilitar su revisión y seguimiento.

Pasos a seguir al crear un Issue:

1. Usar títulos descriptivos que indiquen el problema o la propuesta.
2. Seleccionar la etiqueta correspondiente: Fix, Docs, Error, Question, Update, etc.
3. Describir con detalle el problema, mejora o duda, incluyendo:
4. Comportamiento actual observado
5. Comportamiento esperado
6. Pasos para reproducir (si aplica)
7. Archivos relacionados (si aplica)
8. Capturas de pantalla o logs, si son útiles
9. Asignar el Issue a un miembro del equipo responsable.
10. Asociar el Issue con un hito (milestone) si aplica.
11. Usar checklists cuando se trate de tareas con múltiples pasos.

- 📌 **Objetivo:** Cada Issue debe poder ser reproducido fácilmente o entendido por cualquier colaborador del equipo, permitiendo así un seguimiento eficaz y resolución ordenada.

Crear reportes de error detallados y reproducibles.

Usar plantillas prediseñadas para organizar los issues.

Etiquetar y asignar responsables según prioridad y experiencia.

Pull Requests (PRs)

- Generar desde una rama específica para cada cambio.
- Asociar la PR con un issue usando palabras clave (ej. Closes #12).
- Solicitar revisión antes de fusionar.
- Elegir método de fusión adecuado según política (squash, rebase, merge commit).

Discusiones y Wiki

- Usar GitHub Discussions para tratar propuestas o ideas antes de codificar.
- Registrar decisiones clave y guías en la wiki o en el directorio /docs.

Buenas Prácticas

Usar mensajes de commit descriptivos y consistentes .

Formato de los “commit”

Este es el formato propuesto para los “commit”.

<VCS tag> <type>[ámbito opcional]: <description>

El tipo de confirmación puede incluir lo siguiente:

- feat: se introduce una nueva característica con los cambios
- fix: se ha producido una corrección de errores
- chore: cambios que no se relacionan con una corrección o característica y no modifican src o archivos de prueba (por ejemplo, actualización de dependencias)
- refactor : código refactorizado que no corrige un error ni agrega una característica.
- docs: actualizaciones de la documentación, como el archivo README u otros archivos Markdown.
- style: cambios que no afectan al significado del código, probablemente relacionados con el formato del código, como espacios en blanco, punto y coma que faltan, etc.
- test: incluidas las pruebas nuevas o la corrección de pruebas anteriores

- perf: mejoras de rendimiento
- ci: Relacionado con la integración continua
- build: cambios que afectan al sistema de compilación o a las dependencias externas.
- revert: revierte un “commit” anterior
- La descripción comenzará con la etiqueta de la tarea asignada (alfanumérica) y luego un comentario explícito de los cambios en ese “commit”.

 Integrar con herramientas de comunicación del equipo

Revisar dependencias regularmente con herramientas

Asignar permisos adecuados a cada miembro del equipo.